



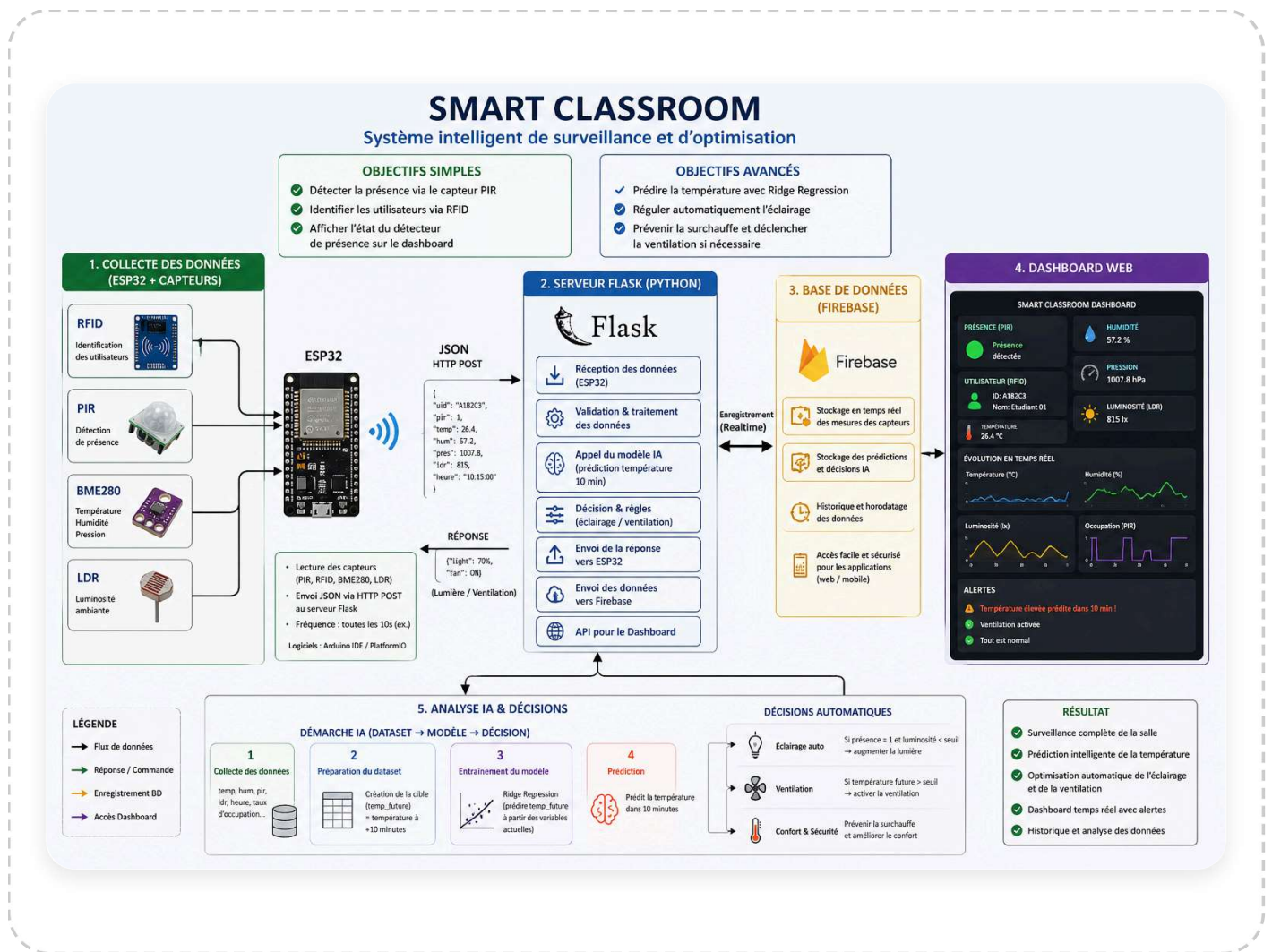
Smart Classroom – Système IoT central

ESP32 · BME280 · PIR · RFID · Flask · Firebase · Prédiction ML · Automatisation de la climatisation

1. Introduction

Le projet **Smart Classroom** fournit un système de surveillance et de contrôle intelligent pour les salles de classe. Il collecte en temps réel des données de température, humidité, mouvement et occupation, les stocke dans Firebase, utilise un modèle d'apprentissage automatique pour prédire la température future, et contrôle automatiquement la climatisation afin d'améliorer le confort et l'efficacité énergétique. Un tableau de bord web affiche les mesures en direct, les statistiques historiques et permet un contrôle manuel de la climatisation.

2. Architecture du système



Composants

- Carte de développement ESP32-WROOM-32
- BME280 (I²C) – température & humidité
- Capteur de mouvement HC-SR501 (PIR)
- Module RFID-RC522 (SPI) – comptage de personnes
- Module relais 1 canal (5V) – contrôle de la climatisation
- Fils Dupont, breadboard, alimentation 5V

Brochages (firmware Arduino)

```
BME280 SDA → GPIO 25
BME280 SCL → GPIO 26
RFID SS    → GPIO 14
RFID MOSI  → GPIO 15
RFID MISO  → GPIO 12
RFID SCK   → GPIO 13
PIR OUT    → GPIO 16
Relais IN  → GPIO 17
```

Reportez-vous au code Arduino (`esp32_firmware.ino`) pour l'implémentation complète.

4. Architecture logicielle

Backend (Flask)

Les principaux points d'entrée fournis par `app.py` :

- `POST /data` – reçoit les mesures des capteurs, déclenche la prédiction et la décision de la clim
- `GET /ac-command` – interrogé par l'ESP32 pour obtenir l'état actuel de la climatisation
- `GET /api/latest` – renvoie les dernières valeurs et l'état de la clim pour le tableau de bord
- `POST /api/set-ac` – point d'accès administrateur pour changer de mode AUTO/MANUEL ou forcer ON/OFF
- `POST /api/set-threshold` – modifie le seuil de température (mode AUTO)
- `GET /api/export-csv` – exporte l'historique des capteurs au format CSV (admin uniquement)

Le serveur Flask maintient un objet `AppState` en mémoire contenant les dernières mesures, la température prédite, l'état de la clim, le mode et le seuil. Un thread d'arrière-plan synchronise les données avec Firebase toutes les 10 secondes.

Tableau de bord frontend

Le tableau de bord (`/dashboard`) est construit avec HTML5, CSS3, JavaScript et Chart.js. Il interroge `/api/latest` toutes les 5 secondes, met à jour les indicateurs (température, humidité, mouvement, nombre de personnes) et le graphique d'évolution de la température. Une boîte « Préviation IA » affiche la température prédite dans 10 minutes. Les widgets de contrôle de la climatisation sont visibles pour les administrateurs.

Structure Firebase Realtime Database

```
{
  "users": { ... },
  "sensors": {
    "latest": { temp, hum, pir, taux_occupation, predicted_temp, timestamp },
    "history": { pushId: { ... mêmes champs } }
  },
  "devices": { "ac": { status, mode } },
  "config": { "ac_threshold": 18.0 }
}
```

5. Pipeline d'apprentissage automatique

Un **SGDRegressor** (descente de gradient stochastique avec erreur quadratique et pénalité L2) est utilisé pour prédire la température 10 minutes à l'avance.

- **Caractéristiques** : température actuelle, humidité, état PIR (0/1), nombre de personnes.
- **Cible** : température mesurée 10 minutes plus tard (`temp_future`).
- **Script d'entraînement** (`train_model.py`) charge trois jeux de données CSV (`data_batch1.csv` ...), normalise les caractéristiques avec `StandardScaler` , et entraîne incrémentalement à l'aide de `partial_fit` .
- **Sortie** : `scaler.pkl` et `temperature_model.pkl` sont sauvegardés et chargés par Flask au démarrage.

Lorsqu'une nouvelle mesure arrive, Flask normalise les caractéristiques, appelle `model.predict()` et stocke le résultat. En mode AUTO, la climatisation s'allume si la température prédite $\geq$ seuil,

sinon elle s'éteint. En mode MANUEL, l'état défini par l'administrateur remplace la décision du modèle.

6. Instructions de déploiement

1. Configuration du backend

- Installez Python 3.9+ et les paquets requis : `pip install flask firebase-admin scikit-learn pandas numpy joblib flask-cors`
- Placez votre fichier `service-account-key.json` (Firebase Admin SDK) à la racine du projet.
- Placez `scaler.pkl` et `temperature_model.pkl` à la racine.
- Exécutez `python app.py` – le serveur démarre sur le port 5000.

2. Configuration matérielle (ESP32)

- Installez les bibliothèques Arduino : WiFi, HTTPClient, Wire, SPI, MFRC522, Adafruit BME280, ArduinoJson.
- Ouvrez `esp32_firmware.ino`, mettez à jour les identifiants WiFi et l'adresse IP du serveur Flask.
- Téléversez le code sur l'ESP32.

3. Test sans matériel (simulateur)

Lancez le script `simulate_sensors.py` sur la même machine que Flask. Il génère des données réalistes et les envoie à `/data` toutes les 5 secondes.

7. Conclusion

Le système Smart Classroom intègre avec succès l'IoT, l'apprentissage automatique et les technologies web en temps réel pour créer un environnement adaptatif. Les améliorations futures pourraient inclure l'ajout de capteurs CO₂, d'alertes par e-mail et d'une application mobile. Le système central actuel constitue une base solide pour des salles de classe économes en énergie et confortables.